

Các Phương pháp Tối ưu hóa Cache

[Đỗ Hữu Khang, Lê Đình Ngọc Thành, Châu Thị Diễm Phúc, 4]

University of Transport HoChiMinh City

1 Giới thiệu

Nhằm vượt qua "bức tường bộ nhớ"(memory wall) giữa CPU và DRAM, tối ưu hóa hệ thống cache là bắt buộc. Bài viết phân tích bốn giải pháp tối ưu hóa trải dài từ quản lý bộ nhớ ảo cấp hệ điều hành, cấu trúc phần cứng đa tầng, cho đến các cơ chế tiền nạp dự đoán và

2 Phần 1: Bộ nhớ ảo và Mở rộng Bộ nhớ bằng SSDAlloc

2.1 Khái niệm Bộ nhớ ảo và Hạn chế của Swap Truyền thống

Bộ nhớ ảo mở rộng không gian địa chỉ vượt quá RAM vật lý bằng cách ánh xạ địa chỉ ảo sang địa chỉ vật lý thông qua bảng trang (Page Table). Tuy nhiên, cơ chế Swap truyền thống của hệ điều hành hoạt động ở mức độ trang cứng nhắc (page granularity - 4KB). Khi xảy ra lỗi trang (Page Fault), OS buộc phải nạp toàn bộ trang 4KB từ SSD vào DRAM ngay cả khi ứng dụng chỉ cần truy cập một đối tượng vài chục byte, gây lãng phí dung lượng DRAM vật lý do nạp kèm các đối tượng "lạnh" (ít sử dụng). Hơn nữa, khi một đối tượng bị thay đổi, OS phải ghi lại cả trang 4KB xuống SSD, dẫn đến hiện tượng khuếch đại ghi (Write Amplification) nghiêm trọng, làm giảm nhanh tuổi thọ và hiệu năng của SSD.

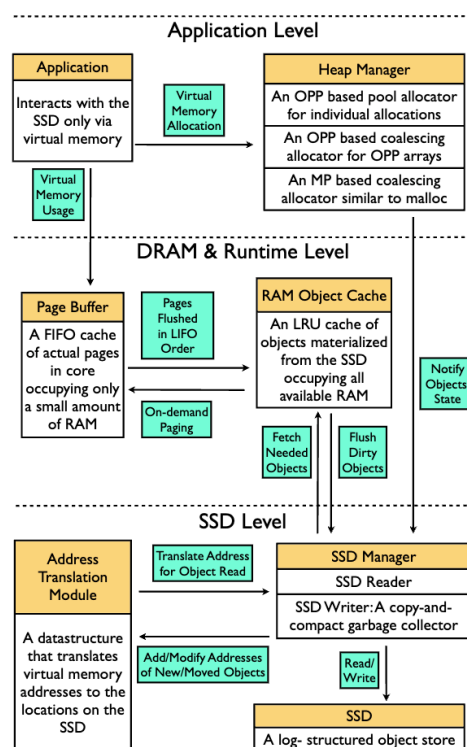
2.2 Kiến trúc và Cơ chế Hoạt động của SSDAlloc so với Swap Truyền thống

Nghiên cứu từ Đại học Princeton đề xuất **SSDAlloc** [1] - trình quản lý bộ nhớ lai DRAM/SSD hoạt động ở mức độ đối tượng (object granularity) nhằm tối ưu hóa hiệu năng và độ bền của bộ nhớ flash. Bằng cách định nghĩa không gian địa chỉ riêng cho từng đối tượng, SSDAlloc thay đổi toàn diện cách thức tương tác với bộ nhớ ảo:

- **Đơn vị quản lý (Granularity):** Swap truyền thống chuyển đổi dữ liệu ở mức trang 4KB thô. SSDAlloc quản lý bộ nhớ ở mức đối tượng (object granularity), cho phép hệ thống chỉ nạp hoặc giải phóng chính xác đối tượng được ứng dụng yêu cầu, giải phóng tối đa băng thông I/O và tối ưu hóa DRAM.
- **Đánh chặn lỗi trang (Page Fault Interception):** Thay vì dựa hoàn toàn vào MMU phần cứng và Kernel để xử lý lỗi trang, SSDAlloc áp dụng mô hình **OPP (One-Page-per-Object)** và lệnh hệ thống mprotect để khóa các trang ảo của đối tượng đã giải phóng xuống SSD. Truy cập bất hợp lệ sẽ kích hoạt tín hiệu SIGSEGV được xử lý ngay tại User

Space bởi thư viện SSDAlloc, dịch địa chỉ nhanh qua cây ATM (Address Translation Map) dạng nhị phân cân bằng thay vì duyệt Page Table của OS.

- **Bộ đệm lai (Hybrid Buffer):** Khác với Swap thông thường chỉ có một bộ đệm trang hệ thống, SSDAlloc chia DRAM thành hai vùng chuyên biệt: *Page Buffer* (cửa sổ trượt chứa các trang thực tế phục vụ CPU xử lý trực tiếp) và *RAM Object Cache* (chứa các đối tượng nén dạng LRU nhằm tối đa hóa số lượng đối tượng lưu trữ trong DRAM vật lý).
- **Quản lý Ghi dữ liệu (Write Management):** Thay vì ghi ngẫu nhiên từng trang 4KB gây tổn hại SSD, SSDAlloc tổ chức ghi dạng log-structured tuần tự. Hệ thống gom cụm (batching) các đối tượng nhỏ dưới 128 byte và căn hàng sector 512 byte trước khi ghi xuống đĩa, triệt tiêu đáng kể hiện tượng Write Amplification.
- **Dọn rác (Garbage Collector):** Thay vì giải phóng trang ngẫu nhiên của Swap thông thường, SSDAlloc sử dụng cơ chế *Copy-and-Compact* tuần tự trên SSD. Việc kiểm tra tính sống sót (liveness) của đối tượng được thực hiện tức thì nhờ tra cứu cấu trúc con trỏ ngược (OTID, OTO) trong Object Table được lưu trực tiếp tại DRAM.



Hình 1: Kiến trúc tổng quan ba tầng của hệ thống SSDAlloc.

3 Phần 2: Cache Đa cấp và ZipCache (Multilevel Caching)

Trong khi SSDAlloc giúp giải quyết hiệu quả bài toán dung lượng, kiến trúc Cache đa cấp kết hợp nén dữ liệu ở Phần 2 sẽ là mảnh ghép tiếp theo để bứt phá giới hạn về tốc độ truy xuất. Xie et al. [2] đề xuất ZipCache - giải pháp hybrid DRAM/SSD tích hợp nén dữ liệu trong suốt.

3.1 Kiến trúc Cache Đa cấp Lai (Hybrid DRAM-SSD Cache)

Mô hình Hybrid DRAM-SSD Cache gồm: **Tầng 1 (DRAM)** tốc độ cực nhanh nhưng đắt đỏ và giới hạn dung lượng; **Tầng 2 (SSD)** dung lượng khổng lồ, giá rẻ nhưng chậm hơn. Mục tiêu là nén dữ liệu để giữ tối đa dữ liệu "nóng" trên RAM, che giấu độ trễ của SSD (ZipCache - Hình 2).

Hình 2: Tổng quan kiến trúc ZipCache.

3.2 Các rào cản kỹ thuật của Cache Đa cấp truyền thống

- **Mất Cục bộ (Locality):** Hash Index phân tán dữ liệu ngẫu nhiên, giảm hiệu quả nén.
- **Khuếch đại Đọc/Ghi (Read/Write Amplification):** Gom các Key-Value nhỏ thành khối 4KB để nén khiến hệ thống phải đọc/ghi toàn khối chỉ để lấy 1 byte.
- **Độ trễ & Tồn CPU:** Nén/giải nén liên tục khối dữ liệu lớn đi ngược lại tiêu chí "độ trễ thấp" của Cache.

3.3 Tối ưu hóa trong ZipCache và Đánh giá

Kiến trúc ZipCache giải quyết triệt để các rào cản thông qua sự kết hợp giữa thuật toán và phần cứng chuyên dụng:

1. **B+ Tree thay cho Hash Index:** Giữ các đối tượng liên kề nằm cạnh nhau, bảo toàn tính cục bộ, tăng tỷ lệ nén gấp đôi.
2. **Kết thúc giải nén sớm (Early Termination):** Node lá chia thành các trang con (sub-pages). CPU định vị và chỉ giải nén một góc nhỏ chứa dữ liệu rồi dừng lại, triệt tiêu độ trễ.
3. **Super-leaf page trên Computational SSD:** Đẩy toàn bộ gánh nặng nén dữ liệu xuống phần cứng SSD chuyên dụng (CSD), tự động nén từng trang 4KB một cách trong suốt.

Đánh giá Hiệu năng: Nhờ tỷ lệ nén cao, Hit Rate trên RAM tăng mạnh. Thử nghiệm cho thấy Throughput tăng tới 72.4% và Latency giảm 42.4% (Hình 3). Đồng thời, việc đẩy tác vụ nén xuống SSD tính toán giúp giảm khuếch đại ghi tới 26.2 lần, bảo vệ và kéo dài tuổi thọ ổ cứng.

Hình 3: So sánh Throughput & Hit Ratio của ZipCache.

Kết luận: ZipCache là minh chứng cho thấy tối ưu Cache đa cấp hiện đại đòi hỏi sự đồng bộ hóa hoàn hảo giữa Cấu trúc chỉ mục phần mềm và Phần cứng chuyên dụng.

4 Phần 3: Nạp trước dữ liệu (Software Prefetching)

Nếu việc tối ưu Cache đa cấp ở Phần 2 mang tính chất phản hồi thụ động, thì cơ chế nạp trước dữ liệu (Prefetching) ở Phần 3 chính là bước tiến chủ động nhằm triệt tiêu hoàn toàn thời gian chờ của bộ vi xử lý.

4.1 Nguyên lý của Prefetching

Dự đoán các dữ liệu hoặc lệnh mà CPU sẽ cần trong tương lai gần và nạp chúng vào cache trước khi chúng thực sự được yêu cầu, giúp giảm thời gian chờ đợi dữ liệu từ bộ nhớ chính, từ đó tăng tốc hiệu suất tổng thể. Hai nguyên lý chính là Spatial Locality và Temporal Locality.

4.2 Phân loại Kỹ thuật Prefetching

1. **Hardware Prefetching:** Việc nạp trước dữ liệu dựa trên phần cứng thường được thực hiện bằng cách sử dụng một cơ chế phần cứng chuyên dụng trong bộ xử lý để theo dõi luồng lệnh hoặc dữ liệu được chương trình đang thực thi yêu cầu, nhận biết một vài phần tử tiếp theo mà chương trình có thể cần dựa trên luồng này và nạp trước chúng vào bộ nhớ cache của bộ xử lý. Các phương pháp Hardware Prefetching: *Stride Prefetcher, Stream Buffer, Irregular Temporal Prefetching*
2. **Software Prefetching:** Chèn các lệnh gợi ý vào mã nguồn. Các lệnh tìm nạp trước ảnh hưởng đến hiệu năng nhưng không ảnh hưởng đến logic chương trình. Việc tìm nạp trước bằng phần mềm có tác động tích cực đến hiệu năng khi được thực hiện đúng thời điểm trước khi tải thực tế. Nếu được thực hiện quá gần thời điểm truy cập, dữ liệu sẽ không được sao chép vào bộ nhớ cache; nếu được thực hiện quá xa, dữ liệu sẽ bị loại bỏ khi thực hiện tải

4.3 Lợi ích và Rủi ro

Lợi ích: Giảm thời gian chờ của CPU, giảm cache miss. Các chỉ số đánh giá Prefetching được kể đến: Coverage, Accuracy, Timeliness, Lateness, Pollution. **Rủi ro:** *Cache Pollution* (làm bẩn cache) nếu dự đoán sai tồn bằng thông bộ nhớ, Prefetch sớm dữ liệu bị đẩy ra khỏi cache trước khi dùng, ngược lại Prefetch muộn sẽ không giảm được độ trễ.

4.4 Độ Tin Cậy Prefetching trên Bộ nhớ Độ Trễ Cao

Prefetch là gợi ý, không bắt buộc. Mahling et al. [3] phát hiện rằng khi bộ nhớ đặt trên các công nghệ khác nhau (DDR, HBM, CXL), khả năng tiên nạp của CPU không đồng đều.

Khi CPU fill buffers (FB) đầy: **Weak Reliability** - loại bỏ prefetch mới, tiếp tục thực thi; **Strong Reliability** - stall đến khi FB trống, đảm bảo prefetch. Đo trên B+-Tree và binary search với dữ liệu local: speedup 1.1-1.9x. Khi remote socket (270ns):

- **AMD EPYC (Weak):** Speedup 1.1-2×, CPU tự drop prefetch thừa.
- **Intel Xeon (Strong):** Speedup 1.3-2.6× nhưng 8KB node = 128 cache lines > 12-line FB = stall, giảm hiệu năng tới 2.5× trên một số kích thước.

Kết luận: Strong tốt local, nguy hiểm remote. Cần tuning động dựa vào vị trí dữ liệu. Với CXL Memory (760ns), CPU cần hỗ trợ tuning prefetch reliability động.

4.5 Ứng dụng trong Web Caching và CDN

Các trình duyệt và CDN tối ưu hoá việc sử dụng tài nguyên, nâng cao hiệu suất bằng cách giảm độ trễ và tăng tốc độ phân phối nội dung, giúp trải nghiệm người dùng tốt hơn.

5 Phần 4: Nén Dữ liệu (Compressed Caching)

Nếu Phần 3 tối ưu hoá tìm nạp chủ động, Phần 4 tối ưu hoá dung lượng bằng nén dữ liệu, cho phép giữ nhiều dữ liệu hơn trong DRAM.

5.1 Động lực và Kiến trúc

Tốc độ CPU tăng 60% hàng năm nhưng độ trễ đĩa chỉ 20%, tạo khoảng trống ngày càng lớn. Compressed Caching tận dụng tính nén được của dữ liệu: chia DRAM thành hai tầng - Uncompressed (dữ liệu nóng, truy cập trực tiếp) và Compressed (dữ liệu ấm, nén để tiết kiệm không gian). Hệ thống tự động di chuyển dữ liệu dựa trên mô hình LRU.

5.2 Các Thuật toán Nén

Wilson et al. [4] so sánh ba thuật toán: **WKdm** (quét 32-bit, từ điển 16 mẫu, tỷ lệ 2:1, độ trễ 0.25ms - tối ưu cho in-memory data); **LZO** (Lempel-Ziv, tỷ lệ cao hơn nhưng chậm 20%); **LZRW1** (Lempel-Ziv đơn giản, kém hiệu quả).

5.3 Thích ứng Động

Hệ thống ước tính: $\text{Speedup} = \frac{T_{\text{Disk}}}{T_{\text{Compress}} + T_{\text{Decompress}}}$. Nếu $\text{Speedup} > 1$, tăng vùng nén; ngược lại, giảm. Kết hợp với Software Prefetch (Phần 3): phần mềm dự đoán và giải nén trước, che giấu độ trễ. Thực nghiệm tiết kiệm 30-70% thời gian paging, tương đương DRAM gấp 2-3 lần dung lượng vật lý.

6 Kết luận

Tối ưu hoá cache hiện đại đòi hỏi sự kết hợp toàn diện giữa bốn công nghệ chính: (1) quản lý bộ nhớ ảo linh hoạt và hỗ trợ SSD (SSDAlloc), (2) kiến trúc cache đa cấp tích hợp nén dữ liệu (ZipCache), (3) nạp trước dữ liệu chủ động với tuning độ tin cậy động (Software Prefetching), và (4) nén dữ liệu thích ứng (Compressed Caching). Các công nghệ này khi được phối hợp hài hòa sẽ tạo nên một "hệ thống bộ nhớ tổng hợp" có khả năng che giấu hoàn toàn độ trễ truy cập, cho phép ứng dụng hiện đại chạy với hiệu suất gần tối ưu dù dữ liệu phải lưu trữ trên các tầng bộ nhớ phức tạp.

Tài liệu

- [1] Anirudh Badam and Vivek S Pai. Ssdalloc: Hybrid ssd/ram memory management made easy. In *NSDI*, volume 11, pages 1–1, 2011.
- [2] Rui Xie, Linsen Ma, Alex Zhong, Feng Chen, and Tong Zhang. ZipCache: A dram/ssd cache with built-in transparent compression. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, pages 1–13, 2024.
- [3] Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. Fetch me if you can: Evaluating cpu cache prefetching and its reliability on high latency memory. In *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*, 2025.
- [4] Paul R Wilson, Scott F Kaplan, and Yannis Smaragdakis. The case for compressed caching in virtual memory systems. In *Proceedings of the USENIX Annual Technical Conference*, pages 519–529, 1999.